

Parking Vision: Google Cloud Platform Deployment Report

Sugakabiyarasha

December 16, 2025

Repository: <https://github.com/Sugakabiyarasha1/parking-vision.git>

Abstract

This report documents the complete deployment of the Parking Vision application to Google Cloud Platform (GCP) using Cloud Run. The application provides an AI-powered parking space detection system with both web-based HTML interface and RESTful API endpoints. The deployment includes deep learning models (SSD and Faster R-CNN) for object detection, integrated into a FastAPI application with Jinja2 templates for server-side rendering.

Contents

1	Introduction	5
1.1	Project Overview	5
1.2	Deployment Objectives	5
2	Architecture	6
2.1	System Architecture	6
2.2	Technology Stack	6
3	Application Structure	6
3.1	Directory Structure	6
3.2	Key Features	7
4	Deployment Configuration	9
4.1	Cloud Run Service Configuration	9
4.2	Docker Configuration	9
4.3	Image Size Optimization	9
4.4	Environment Variables	10
5	API Endpoints	10
5.1	HTML Endpoints	10
5.2	JSON API Endpoints	10

6	Model Information	10
6.1	Available Models	10
6.2	Model Loading Strategy	11
7	Deployment Process	11
7.1	Prerequisites	11
7.2	Deployment Steps	11
7.3	Deployment Commands	12
8	Testing and Validation	13
8.1	Local Testing	13
8.2	Post-Deployment Testing	13
8.3	Postman Collection	14
8.4	Application Testing Results	14
9	Performance Considerations	22
9.1	Cold Start	22
9.2	Resource Usage	22
9.3	Scaling	23
10	Security Considerations	23
10.1	Current Configuration (POC)	23
10.2	Production Recommendations	23
11	Cost Analysis	23
11.1	Pricing Model	23
11.2	Estimated Costs	23
12	Challenges and Solutions	24
12.1	Challenges Encountered	24
13	Lessons Learned	24
14	Future Enhancements	24
14.1	Recommended Improvements	24
15	Deployment Results	25
15.1	Final Deployment Status	25
15.2	Service Endpoints	26
15.3	Optimization Achievements	26
16	Conclusion	26
17	References	26

List of Figures

1	Machine Learning Canvas - Project Overview	5
2	Web Application Home Page - Upload Interface	8
3	Cloud Build History - Build Pipeline Overview	12
4	Build Summary - Successful Image Build (Version 2.0)	12
5	Artifact Registry - Docker Image Repository	12
6	Postman API Test Results - Successful Endpoint Testing	14
7	Image Upload Interface - SSD Model Selection	15
8	SSD Model Detection Results - Example 1	16
9	SSD Model Detection Results - Example 2	17
10	SSD Model Detection Results - Example 3	18
11	Image Upload Interface - Faster R-CNN Model Selection	19
12	Faster R-CNN Model Detection Results - Example 1	20
13	Faster R-CNN Model Detection Results - Example 2	21
14	Faster R-CNN Model Detection Results - Example 3	22
15	Cloud Run Services Dashboard - Service List	25
16	Cloud Run Service Overview - parking-vision-web Details	25

List of Tables

1	Technology Stack	6
2	Cloud Run Configuration	9
3	Image Size Optimization Results	10
4	HTML Endpoints	10
5	JSON API Endpoints	10
6	Final Deployment Information	25

1 Introduction

1.1 Project Overview

Parking Vision is a deep learning-based system designed to detect and classify parking spaces in images. The system utilizes state-of-the-art object detection models to identify free and occupied parking spaces, providing real-time analysis capabilities through a web interface and programmatic API access.

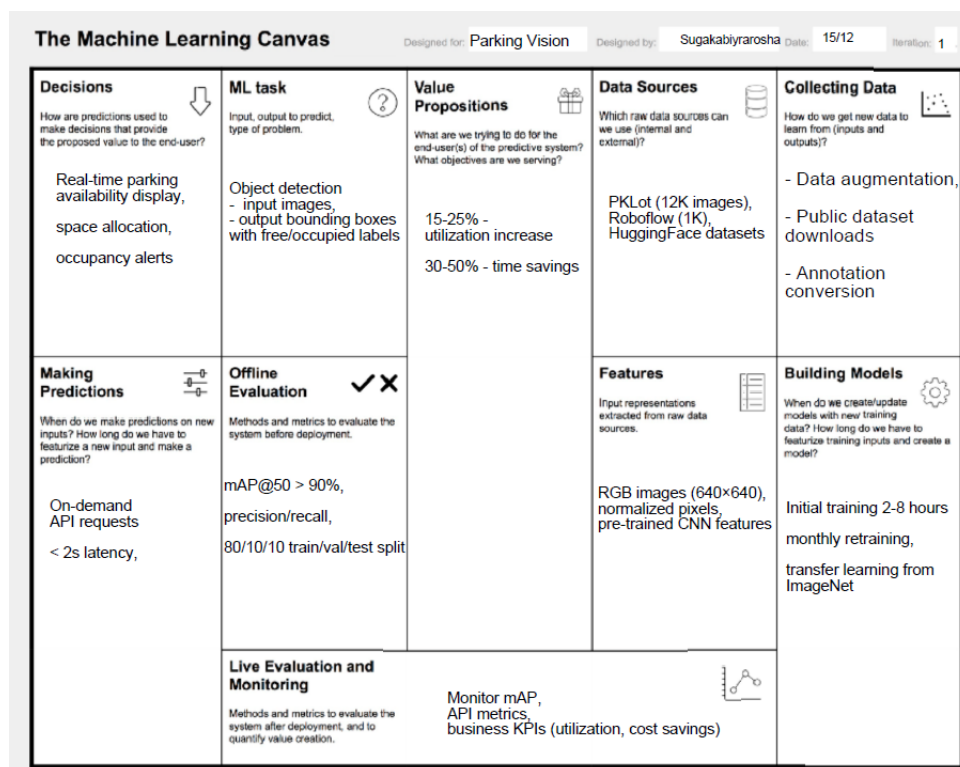


Figure 1: Machine Learning Canvas - Project Overview

1.2 Deployment Objectives

The primary objectives of this GCP deployment were:

- Deploy the application as a single Cloud Run service
- Provide both HTML web interface and JSON API endpoints
- Ensure model checkpoints are included in the container image
- Optimize container size for faster deployment
- Enable public access without authentication (POC phase)
- Document the complete deployment process

2 Architecture

2.1 System Architecture

The deployed application follows a monolithic architecture with the following components:

- **Frontend:** Server-rendered HTML using Jinja2 templates
- **Backend:** FastAPI application with RESTful API endpoints
- **Models:** SSD and Faster R-CNN detection models (baked into container)
- **Infrastructure:** Google Cloud Run (serverless container platform)

2.2 Technology Stack

Component	Technology
Web Framework	FastAPI 0.104.1
Template Engine	Jinja2 3.1.2+
ASGI Server	Uvicorn 0.24.0
Deep Learning	PyTorch 2.0.0+cpu (CPU-only, optimized)
Computer Vision	OpenCV 4.8.0+ (headless)
Image Processing	Pillow 10.0.0+
Container Platform	Docker + Cloud Run

Table 1: Technology Stack

3 Application Structure

3.1 Directory Structure

The application follows a modular structure:

```
GCP_deployment/  
|-- app.py                # Main FastAPI application  
|-- Dockerfile            # Container definition  
|-- requirements.txt      # Python dependencies  
|-- models/               # Model architectures and loaders  
|-- inference/            # Inference functions  
|-- utils/                # Utility functions  
|-- templates/            # Jinja2 HTML templates  
|   |-- index.html        # Upload form  
|   '-- result.html       # Results display  
|-- static/               # Static files (CSS, JS)  
|   |-- css/  
|   '-- js/  
'-- checkpoints/          # Model weights  
    |-- phase2_ssd_parking/  
    '-- phase2_faster_rcnn_parking/
```

3.2 Key Features

1. **HTML Web Interface:** User-friendly upload form with model selection and confidence threshold adjustment
2. **Results Visualization:** Annotated images with bounding boxes and detection statistics
3. **RESTful API:** JSON endpoints for programmatic access (Postman testing)
4. **Model Management:** On-demand model loading with in-memory caching
5. **Health Monitoring:** Health check endpoints for service monitoring

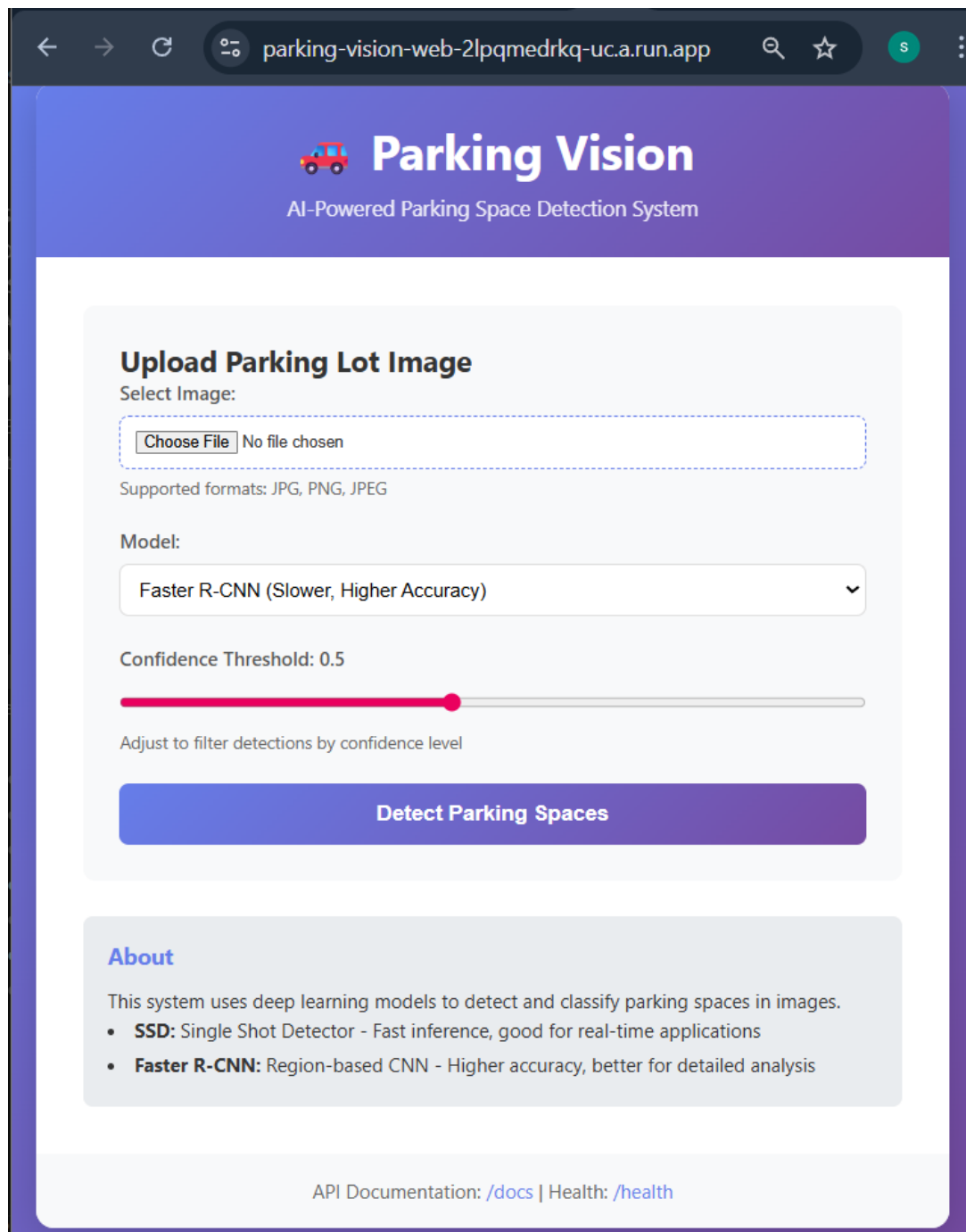


Figure 2: Web Application Home Page - Upload Interface

4 Deployment Configuration

4.1 Cloud Run Service Configuration

Parameter	Value
Service Name	parking-vision-web
Region	us-central1
Platform	Managed
Memory	4 GiB
CPU	2 vCPU
Timeout	300 seconds (5 minutes)
Port	8080 (via PORT env var)
Min Instances	0 (scales to zero)
Max Instances	10
Authentication	Public (unauthenticated)

Table 2: Cloud Run Configuration

4.2 Docker Configuration

The Dockerfile implements the following optimizations:

- **Multi-stage build:** Two-stage build (builder + runtime) to reduce image size
- **CPU-only PyTorch:** Using `torch==2.0.0+cpu` instead of full version (saves 6GB)
- **Minimal dependencies:** Removed unused packages (pandas, scikit-learn, matplotlib, seaborn)
- **OpenCV headless:** Using `opencv-python-headless` for smaller footprint
- System dependencies for OpenCV (libgl1, libglib2.0-0, etc.)
- Python 3.10 slim base image
- Model checkpoints included in image (only `best_model.pt` files)
- Proper port binding (`0.0.0.0:$PORT`)

4.3 Image Size Optimization

The container image was optimized from 9.17GB to 2.03GB (77% reduction) through:

Component	Before	After	Savings
PyTorch (full)	6GB	-	-
PyTorch (CPU-only)	-	500MB	5.5GB
Unused packages	500MB	-	500MB
Multi-stage build	-	Optimized	200MB
Total	9.17GB	2.03GB	7.14GB

Table 3: Image Size Optimization Results

4.4 Environment Variables

The application uses the following environment variables:

- **PORT**: Port number (default: 8080, set by Cloud Run)
- **DEVICE**: Computation device (default: cpu)

5 API Endpoints

5.1 HTML Endpoints

Method	Path	Description
GET	/	Upload form (index.html)
POST	/detect	Process image and render results

Table 4: HTML Endpoints

5.2 JSON API Endpoints

Method	Path	Description
GET	/health	Health check status
GET	/models	List available models
GET	/models/{name}	Get model details
POST	/models/load	Load model into memory
POST	/api/predict	Predict parking spaces (JSON)
GET	/docs	Interactive API documentation

Table 5: JSON API Endpoints

6 Model Information

6.1 Available Models

The system includes two detection models:

1. **SSD (Single Shot Detector)**

- Checkpoint: `phase2_ssd_parking/best_model.pt`
- Size: 43 MB
- Characteristics: Fast inference, suitable for real-time applications

2. Faster R-CNN

- Checkpoint: `phase2_faster_rcnn_parking/best_model.pt`
- Size: 472 MB
- Characteristics: Higher accuracy, better for detailed analysis

6.2 Model Loading Strategy

Models are loaded on-demand when first requested, then cached in memory for subsequent requests. This approach:

- Reduces cold start time
- Optimizes memory usage
- Allows switching between models dynamically

7 Deployment Process

7.1 Prerequisites

1. Google Cloud Platform account with billing enabled
2. gcloud CLI installed and authenticated
3. Required APIs enabled:
 - Cloud Run API
 - Artifact Registry API
 - Cloud Build API
4. Docker Desktop (for local testing)

7.2 Deployment Steps

1. **Local Testing:** Test application in Docker container
2. **Build Image:** Use Cloud Build to create container image
3. **Push to Registry:** Store image in Artifact Registry
4. **Deploy Service:** Deploy to Cloud Run with specified configuration
5. **Verify Deployment:** Test all endpoints and functionality

Status	Build	Region	Source	Ref	Commit	Trigger Name	Created	Duration	Security Insights
✓	93f36770	global	—	—	—	—	12/16/25, 6:47 AM	5 min 4 sec	View
✓	35e3a566	global	—	—	—	—	12/16/25, 6:29 AM	9 min 19 sec	View
✓	76e7d7c5	global	—	—	—	—	12/16/25, 5:49 AM	13 min 16 sec	View
✗	19334696	global	—	—	—	—	12/16/25, 5:37 AM	6 min 45 sec	—
✗	dc5375c8	global	—	—	—	—	12/16/25, 5:30 AM	37 sec	—
✓	a968d4c5	global	—	—	—	—	12/16/25, 1:33 AM	5 min 29 sec	View
✓	367b7296	global	—	—	—	—	12/16/25, 12:50 AM	6 min 15 sec	View

Figure 3: Cloud Build History - Build Pipeline Overview

Steps	Status	Build	Duration
Build Summary	✓	93f36770-1d21-4795-a2f1-e45bfd422336	00:05:04
0: gcr.io/cloud-builders/docker build --network cloudbuild --no-cache ...	✓		00:03:00

Build Summary
<pre> 330 Preparing to unpack .../47-x11proto-dev_2024.1-1_all.deb ... 331 Unpacking x11proto-dev (2024.1-1) ... 332 Selecting previously unselected package libxau-dev:amd64. 333 Preparing to unpack .../48-libxau-dev_1:1.0.11-1_amd64.deb ... 334 Unpacking libxau-dev:amd64 (1:1.0.11-1) ... 335 Selecting previously unselected package libxdmcp-dev:amd64. 336 Preparing to unpack .../49-libxdmcp-dev_1:1.1.5-1_amd64.deb ... 337 Unpacking libxdmcp-dev:amd64 (1:1.1.5-1) ... 338 Selecting previously unselected package xtrans-dev. 339 Preparing to unpack .../50-xtrans-dev_1.4.0-1_all.deb ... 340 Unpacking xtrans-dev (1.4.0-1) ... 341 Selecting previously unselected package libxcb1-dev:amd64. 342 Preparing to unpack .../51-libxcb1-dev_1.17.0-2+b1_amd64.deb ... 343 Unpacking libxcb1-dev:amd64 (1:1.17.0-2+b1) ... </pre>

Figure 4: Build Summary - Successful Image Build (Version 2.0)

Repository Details
Format: Docker Type: Standard Tags: — Scanning enablement: ⚠ Scanning inactive (Scanning API disabled) Show less

Name	Connection	Created	Updated
parking-vision-web	—	3 hours ago	2 hours ago

Figure 5: Artifact Registry - Docker Image Repository

7.3 Deployment Commands

```

# Set variables
PROJECT_ID="parking-vision-deployment"
REGION="us-central1"
REPO="parking-vision"
SERVICE="parking-vision-web"

# Build and push (Version 2.0 - Optimized)
gcloud builds submit \
  --tag $REGION-docker.pkg.dev/$PROJECT_ID/$REPO/$SERVICE:2.0 .

# Deploy
gcloud run deploy $SERVICE \
  --image $REGION-docker.pkg.dev/$PROJECT_ID/$REPO/$SERVICE:2.0 \
  --allow-unauthenticated \
  --cpu 2 --memory 4Gi \
  --timeout 300 \
  --port 8080 \
  --region us-central1

```

8 Testing and Validation

8.1 Local Testing

Before deployment, the application was tested:

- Locally without Docker (development mode)
- In Docker container (production simulation)
- All HTML routes verified
- All API endpoints tested
- Model loading and inference validated

8.2 Post-Deployment Testing

After deployment, the following tests were performed:

1. Health check endpoint (/health)
2. HTML frontend accessibility (/)
3. Image upload and processing (/detect)
4. API endpoint via Postman (/api/predict)
5. Model switching functionality
6. Error handling and edge cases

8.3 Postman Collection

A complete Postman collection has been created for API testing, including:

- Pre-configured requests for all endpoints
- Test scripts with assertions
- Environment variables for easy URL management
- Error handling for common issues (422 validation errors)

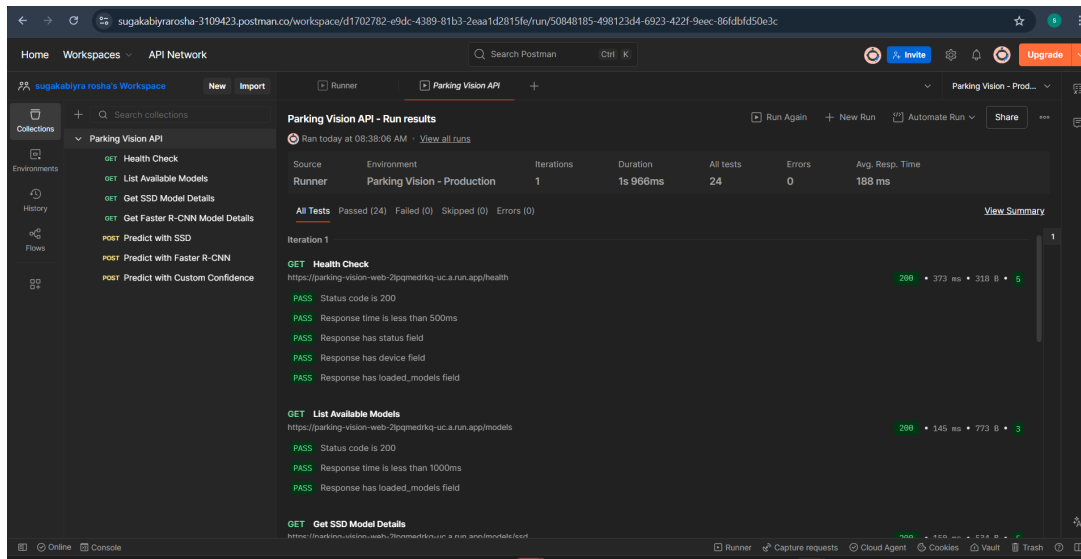


Figure 6: Postman API Test Results - Successful Endpoint Testing

Postman Team Invite: https://app.getpostman.com/join-team?invite_code=a328a37becb22e56c6d1761882dfe44ecdfe85e3b0cabcd6f60493094557e0b&target_code=a411d34d08f5940b1a794a55dd7ebdc6

8.4 Application Testing Results

The following figures demonstrate the application's functionality with both SSD and Faster R-CNN models:



Parking Vision

AI-Powered Parking Space Detection System

Upload Parking Lot Image

Select Image:

Choose File

img_035532.jpg

Supported formats: JPG, PNG, JPEG

Model:

SSD (Faster, Lower Accuracy) ▾

Confidence Threshold: 0.5

Adjust to filter detections by confidence level

Detect Parking Spaces

About

This system uses deep learning models to detect and classify parking spaces in images.

- **SSD**: Single Shot Detector - Fast inference, good for real-time applications
- **Faster R-CNN**: Region-based CNN - Higher accuracy, better for detailed analysis

Figure 7: Image Upload Interface - SSD Model Selection



Parking Vision - Results

← Upload New Image

Detection Statistics

Model Used:

SSD

Total Detections:

37

Free Spaces:

15

Occupied Spaces:

22

Confidence Threshold:

0.5

Processing Time:

1.69s

Annotated Image



Figure 8: SSD Model Detection Results - Example 1

Annotated Image



Detection Details

#	Class	Confidence	Bounding Box
1	not_free_parking_space	1.00	[562, 186, 622, 246]
2	not_free_parking_space	1.00	[372, 414, 449, 499]

Figure 9: SSD Model Detection Results - Example 2

23	free_parking_space	1.00	[298, 272, 367, 345]
24	free_parking_space	1.00	[186, 206, 253, 266]
25	not_free_parking_space	1.00	[173, 32, 224, 69]
26	not_free_parking_space	1.00	[605, 243, 640, 308]
27	free_parking_space	1.00	[209, 253, 281, 330]
28	free_parking_space	1.00	[207, 94, 264, 140]
29	free_parking_space	1.00	[488, 178, 533, 256]
30	free_parking_space	1.00	[412, 98, 459, 161]
31	free_parking_space	1.00	[126, 52, 178, 93]
32	not_free_parking_space	1.00	[422, 34, 476, 79]
33	free_parking_space	1.00	[149, 116, 210, 167]
34	not_free_parking_space	1.00	[150, 4, 201, 42]
35	free_parking_space	1.00	[372, 0, 423, 21]
36	not_free_parking_space	1.00	[221, 127, 288, 180]
37	free_parking_space	1.00	[186, 53, 248, 104]

Download Results

Download JSON

Download Image

Figure 10: SSD Model Detection Results - Example 3

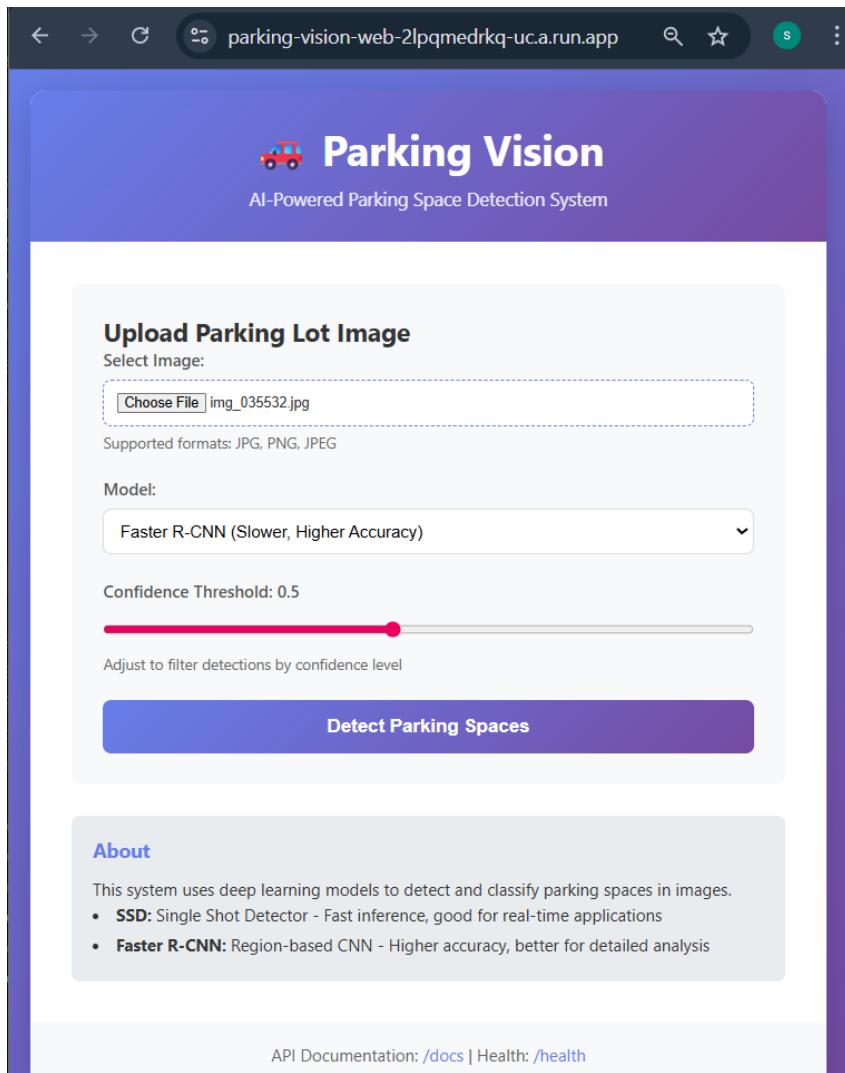


Figure 11: Image Upload Interface - Faster R-CNN Model Selection

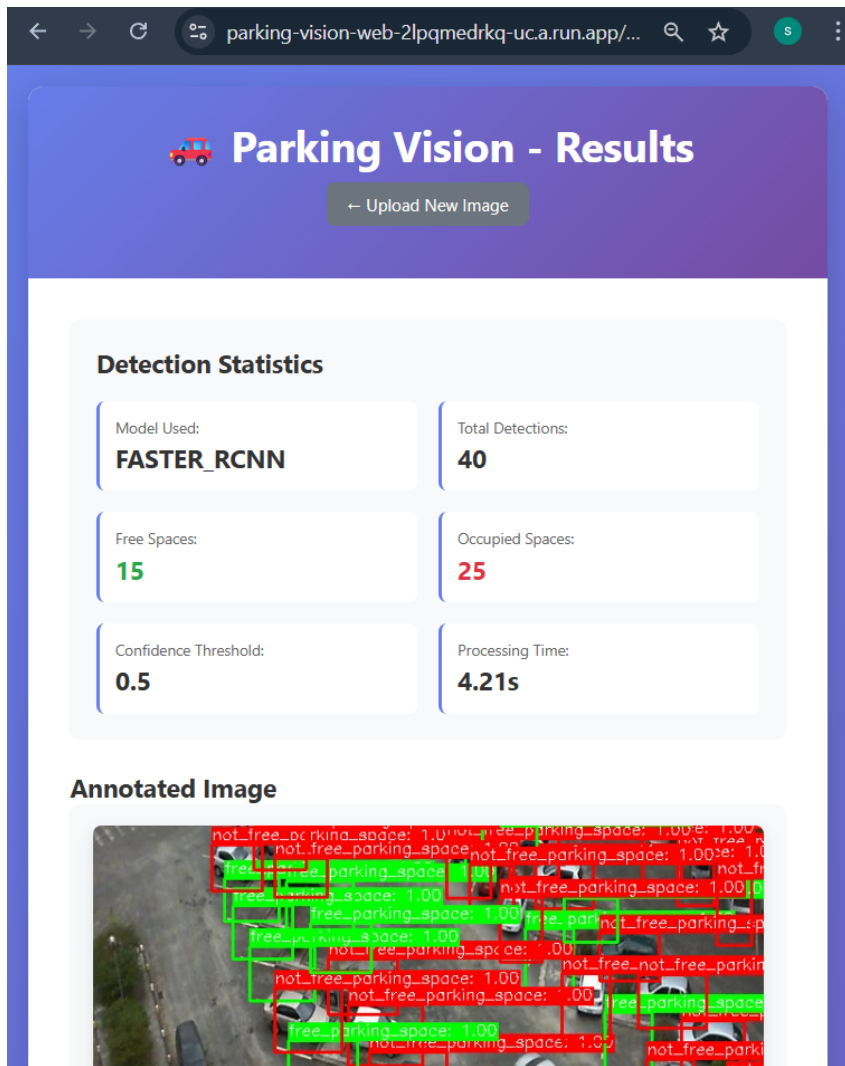


Figure 12: Faster R-CNN Model Detection Results - Example 1



Figure 13: Faster R-CNN Model Detection Results - Example 2

26	not_free_parking_space	1.00	[301, 460, 397, 578]
27	not_free_parking_space	1.00	[596, 50, 629, 114]
28	not_free_parking_space	1.00	[528, 223, 574, 297]
29	not_free_parking_space	1.00	[447, 141, 494, 206]
30	not_free_parking_space	1.00	[398, 12, 444, 50]
31	not_free_parking_space	1.00	[337, 13, 380, 70]
32	not_free_parking_space	1.00	[371, 414, 448, 501]
33	free_parking_space	1.00	[372, 0, 423, 21]
34	not_free_parking_space	1.00	[521, 143, 585, 194]
35	not_free_parking_space	1.00	[173, 32, 223, 69]
36	not_free_parking_space	1.00	[388, 68, 429, 131]
37	not_free_parking_space	1.00	[174, 155, 238, 216]
38	not_free_parking_space	1.00	[359, 38, 402, 92]
39	not_free_parking_space	1.00	[153, 4, 203, 42]
40	free_parking_space	1.00	[208, 94, 265, 140]

Download Results

[Download JSON](#)

[Download Image](#)

Figure 14: Faster R-CNN Model Detection Results - Example 3

9 Performance Considerations

9.1 Cold Start

- First request may take 10-15 seconds (model loading)
- Subsequent requests are faster (models cached in memory)
- Can be mitigated with `-min-instances 1` (increases cost)

9.2 Resource Usage

- Memory: 1-2 GB during inference (depends on model)
- CPU: Efficient utilization with 2 vCPU
- Timeout: 300 seconds sufficient for model loading + inference

9.3 Scaling

- Automatic scaling based on traffic
- Scales to zero when idle (cost optimization)
- Maximum 10 instances under load

10 Security Considerations

10.1 Current Configuration (POC)

- Public access enabled (`-allow-unauthenticated`)
- No authentication required
- Suitable for demonstration and testing

10.2 Production Recommendations

For production deployment, considering:

- Enabling Cloud IAM authentication
- Implementing rate limiting
- Adding request validation and sanitization
- Enabling Cloud Armor for DDoS protection
- Using HTTPS only (enforced by Cloud Run)
- Implementing API key authentication for programmatic access

11 Cost Analysis

11.1 Pricing Model

Cloud Run pricing is based on:

- Request count (first 2 million requests/month free)
- CPU time (first 360,000 GB-seconds/month free)
- Memory time (first 180,000 vCPU-seconds/month free)

11.2 Estimated Costs

For typical usage:

- Free tier covers most POC usage
- After free tier: \$0.00002400 per request
- With 300+ credits: Sufficient for extended testing

12 Challenges and Solutions

12.1 Challenges Encountered

1. **Port Binding:** Initial configuration used hardcoded port
 - Solution: Updated to use `$PORT` environment variable
2. **Host Binding:** Application bound to localhost
 - Solution: Changed to `0.0.0.0` for Cloud Run compatibility
3. **Template Integration:** FastAPI template support
 - Solution: Used Starlette's Jinja2Templates integration
4. **Image Size:** Large container image (9.17GB)
 - Solution: Multi-stage build + CPU-only PyTorch (reduced to 2.03GB, 77% reduction)
5. **Template Attribute Error:** Missing 'confidence' attribute in detections
 - Solution: Added 'confidence' as alias for 'score' in detection dictionaries
6. **PATH Configuration:** uvicorn command not found in multi-stage build
 - Solution: Used 'python -m uvicorn' instead of direct 'uvicorn' command

13 Lessons Learned

1. Cloud Run requires strict adherence to container contract (PORT, 0.0.0.0)
2. Model loading strategy significantly impacts cold start time
3. Jinja2 templates provide clean separation of concerns
4. Local Docker testing is essential before cloud deployment
5. Proper error handling improves user experience

14 Future Enhancements

14.1 Recommended Improvements

Considering the following enhancements:

1. **Authentication:** Implementing user authentication
2. **Caching:** Adding Redis for result caching
3. **Monitoring:** Integrating Cloud Monitoring and Logging

4. **Optimization:** Further reducing container image size (currently 2.03GB, achieved 77% reduction)
5. **CI/CD:** Automating deployment with Cloud Build triggers
6. **Custom Domain:** Mapping custom domain to Cloud Run service
7. **Load Testing:** Performing comprehensive load testing

15 Deployment Results

15.1 Final Deployment Status

The application was successfully deployed to Google Cloud Run with the following details:

Parameter	Value
Service Name	parking-vision-web
Service URL	https://parking-vision-web-2lpqmedrkq-uc.a.run.app
Project ID	parking-vision-deployment
Region	us-central1
Image Version	2.0 (optimized)
Image Size	2.03GB (optimized from 9.17GB)
Build Duration	5 minutes (optimized)
Deployment Status	Successfully Deployed

Table 6: Final Deployment Information

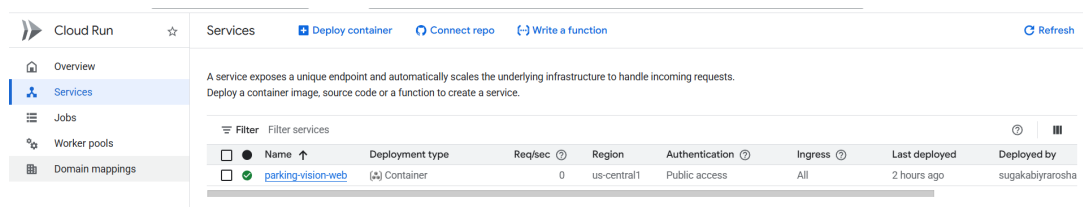


Figure 15: Cloud Run Services Dashboard - Service List

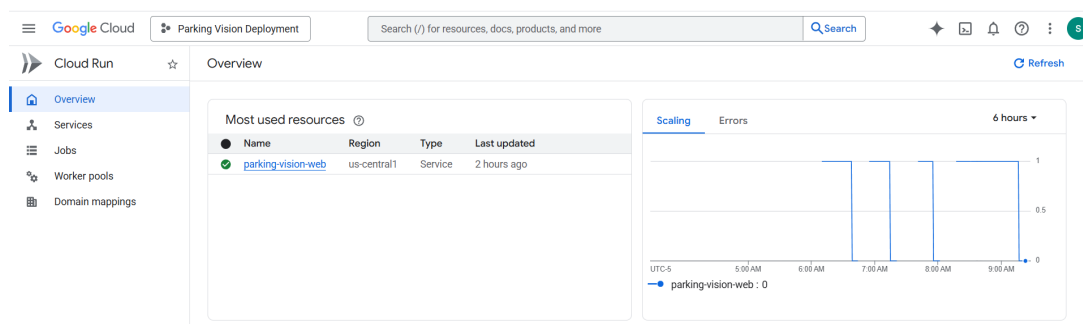


Figure 16: Cloud Run Service Overview - parking-vision-web Details

15.2 Service Endpoints

The deployed service provides the following publicly accessible endpoints:

- **Frontend:** <https://parking-vision-web-2lpqmedrkq-uc.a.run.app/>
- **Health Check:** <https://parking-vision-web-2lpqmedrkq-uc.a.run.app/health>
- **Models List:** <https://parking-vision-web-2lpqmedrkq-uc.a.run.app/models>
- **API Endpoint:** <https://parking-vision-web-2lpqmedrkq-uc.a.run.app/api/predict>

15.3 Optimization Achievements

1. **Image Size Reduction:** Reduced from 9.17GB to 2.03GB (77% reduction, 7.14GB saved)
2. **Build Time:** Reduced from 13 minutes to 5 minutes
3. **Cold Start:** Improved due to smaller image size
4. **Cost Efficiency:** Lower storage and transfer costs

16 Conclusion

The Parking Vision application has been successfully deployed to Google Cloud Platform using Cloud Run. The deployment provides both a user-friendly web interface and programmatic API access, making it suitable for various use cases. The system demonstrates effective integration of deep learning models with cloud infrastructure, following best practices for containerized deployments.

Through optimization efforts, the container image size was reduced by 77%, significantly improving deployment speed and reducing costs. The application is now publicly accessible at <https://parking-vision-web-2lpqmedrkq-uc.a.run.app> and ready for demonstration, testing, and further development. The modular architecture allows for easy extension and enhancement as requirements evolve.

17 References

- Google Cloud Run Documentation: <https://cloud.google.com/run/docs>
- FastAPI Documentation: <https://fastapi.tiangolo.com>
- Cloud Run Container Contract: <https://cloud.google.com/run/docs/container-contract>
- PyTorch Documentation: <https://pytorch.org/docs>